

20.多输入模型

1. 介绍

多输入神经网络是一种能够同时处理多个**不同来源、不同模态或不同特征**的数据输入的深度学习模型架构。与传统的单输入模型不同，多输入模型可以融合多种信息源，充分利用数据的互补性，从而提高模型的性能和鲁棒性。

在RKNN框架中，多输入模型的典型应用场景包括：

- 1. **双输入模型**：如Siamese网络，接收两个图像作为输入进行对比。
- 2. **图像+文本模型**：如CLIP模型，同时接收图像和文本输入。
- 3. **多模态模型**：接收不同模态的数据，如音频和视频。
- 4. **特征融合模型**：接收多个特征向量作为输入。

2. 多输入与多batch区别

区别维度	多输入模型 (Multi-input)	多Batch模型 (Multi-batch)
核心目的	模型需要多种数据才能工作	为了让模型跑得更快、处理量更大
输入数据结构	异构数据：[数据A, 数据B, 数据C]	同构数据打包：(N, 高, 宽, 通道)
数据关系	数据之间互补（比如图+文字）	数据之间并列（比如8张不同的狗图）

总结一下：

- 多输入是问模型：“我给你这几样东西，你能算出结果吗？”
- 多Batch是问模型：“我一次性给你这么多份，你能快点算完吗？”

3. 代码测试

11_multi_input

如果遇到找不到包，需要先进行下载。

代码块

```
1 pip3 install 'tensorflow>=1.12.0,<=2.14.0' -i
```

代码块

```
1  import numpy as np
2  import cv2
3  from rknn.api import RKNN
4
5  if __name__ == '__main__':
6
7      # Create RKNN object
8      rknn = RKNN(verbose=True)
9
10     # Pre-process config (配置模型预处理参数：设置均值和方差，并指定目标硬件平台为
    RK3588)
11     print('--> Config model')
12     rknn.config(mean_values=[[127.5, 127.5, 127.5], [0, 0, 0], [0, 0, 0],
    [127.5]],
13                 std_values=[[128, 128, 128], [1, 1, 1], [1, 1, 1], [128]],
14                 target_platform='rk3588')
15     print('done')
16
17     # Load model (加载TensorFlow模型：指定输入输出节点名称以及四个输入张量的具体形状)
18     print('--> Loading model')
19     ret = rknn.load_tensorflow(tf_pb='./conv_128.pb',
20                                inputs=['input1', 'input2', 'input3', 'input4'],
21                                outputs=['output'],
22                                input_size_list=[[1, 128, 128, 3], [1, 128,
    128, 3], [1, 128, 128, 3], [1, 128, 128, 1]])
23     if ret != 0:
24         print('Load model failed!')
25         exit(ret)
26     print('done')
27
28     # Build model (构建模型：开启INT8量化，并使用dataset.txt中的数据进行校准)
29     print('--> Building model')
30     ret = rknn.build(do_quantization=True, dataset='./dataset.txt')
31     if ret != 0:
32         print('Build model failed!')
33         exit(ret)
34     print('done')
35
36     # Export rknn model (导出编译好的RKNN模型文件)
37     print('--> Export rknn model')
38     ret = rknn.export_rknn('./conv_128.rknn')
39     if ret != 0:
```

```

40         print('Export rknn model failed!')
41         exit(ret)
42     print('done')
43
44     # Init runtime environment (初始化运行时环境, 准备执行推理)
45     print('--> Init runtime environment')
46     ret = rknn.init_runtime()
47     if ret != 0:
48         print('Init runtime environment failed!')
49         exit(ret)
50     print('done')
51
52     # Set inputs (准备多路输入数据: 读取图片、转换颜色空间、调整维度以匹配模型输入要求)
53     img = cv2.imread('./dog_128x128.jpg')
54     img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)          # nhwc
55     img = np.expand_dims(img, 0)
56
57     img_gray = cv2.imread('./dog_128x128_gray.png', cv2.IMREAD_GRAYSCALE)
58     img_gray = np.expand_dims(img_gray, -1)
59     img_gray = np.expand_dims(img_gray, 0)             # nhwc
60
61     input2 = np.load('input2.npy').astype('float32')   # nchw
62
63     input3 = np.load('input3.npy').astype('float32')   # nchw
64
65     # Inference (执行模型推理: 传入四路不同格式的数据, 并分别指定其排布方式nhwc或nchw)
66     print('--> Running model')
67     outputs = rknn.inference(inputs=[img, input2, input3, img_gray],
data_format=['nhwc', 'nchw', 'nchw', 'nhwc'])
68     # np.save('./functions_multi_input_0.npy', outputs[0])
69     print('done')
70     outputs[0] = outputs[0].reshape((1, -1))
71     print('inference result: ', outputs)
72
73     print('--> 开始将输出结果转换为可视化的热力图...')
74
75     # 1. 取出输出结果, 并还原成 128x128 的图片形状
76     feature_map = outputs[0].reshape(128, 128)
77
78     # 2. 归一化处理: 把一堆杂乱的数字 (比如 -15 到 -3) 拉伸到 0~255 的像素范围
79     # 找出最大值和最小值
80     f_min = feature_map.min()
81     f_max = feature_map.max()
82
83     # 映射到 0-255 区间, 并转为 uint8 格式 (OpenCV 图片的标准格式)
84     if f_max > f_min:
85         heatmap = (feature_map - f_min) / (f_max - f_min) * 255

```

```

86         else:
87             heatmap = np.zeros_like(feature_map)
88             heatmap = heatmap.astype(np.uint8)
89
90             # 3. 为了看得更清楚，给它上个颜色（应用 COLORMAP_JET 颜色映射，越亮的地方越红）
91             heatmap_color = cv2.applyColorMap(heatmap, cv2.COLORMAP_JET)
92
93             # 4. 保存并展示这张“特征热力图”
94             cv2.imwrite('output_heatmap.jpg', heatmap_color)
95             print('done! 请在当前目录下查看 output_heatmap.jpg')
96             # =====
97
98             rknn.release()

```

[查看结果](#)

```
done
--> Init runtime environment
I Target is None, use simulator!
done
--> Running model
I GraphPreparing : 100%|██████████████████████████████████████| 13/13 [00:00<00:00, 12160.11it/s]
I SessionPreparing : 100%|██████████████████████████████████████| 13/13 [00:00<00:00, 4532.87it/s]
done
inference result: [array([[ -3.1041162,  -7.0548096,  -5.361655 , ..., -14.391811 ,
        -15.802773 , -11.287695 ]], dtype=float32)]
--> 开始将输出结果转换为可视化的热力图...
done! 请在当前目录下查看 output_heatmap.jpg
(rknn-toolkit2) topeet@ubuntu:~/rknn/04_code/11_multi_input$
```

输入模型结构

